

Report for the Course on Cyber-Physical Systems and IoT Security

Title of the reference paper: P. Huitsing, R. Chandia, M. Papa and S. Shenoi, "Attack taxonomies for the Modbus protocols," International Journal of Critical Infrastructure Protection, vol. 1, pp. 37–44, 2008.

Name of who is involved in the project: Saeid Abbasnejad — Student No. 2154285

Objectives

Introduction. Modbus is one of the oldest and most widely deployed industrial control protocols; in the oil and gas sector it is the predominant protocol for pipeline operations. It was designed in 1979 for trusted serial links and carries *no authentication, no authorisation, no integrity protection and no encryption*. The reference paper by Huitsing *et al.* performs a systematic security analysis of the Modbus Serial and Modbus TCP protocols and organises the results into two **attack taxonomies**. It classifies every attack along two axes — the *threat category* (interception, interruption, modification, fabrication) and the *targeted asset* (master, field device, communication link / network path, message) — identifying 20 distinct attacks (59 instances) for Modbus Serial and 28 distinct attacks (113 instances) for Modbus TCP.

The reference paper is purely analytical: it enumerates and classifies attacks but reports *no experiments*. The agreed objective of this project is therefore to turn the taxonomy into a **practical, self-contained reproduction** — a live Modbus TCP master/slave testbed with a small simulated physical process — and to actually carry out a representative attack from every threat category, measuring its real effect. Concretely we set out to:

- implement a working Modbus TCP stack (slave/server, master/client and the wire protocol) from the specification, with no industrial hardware;
- recreate normal master–slave communication controlling a simulated tank/pumping process, as an honest baseline;
- implement and run representative attacks named in the taxonomy — reconnaissance (B6/S5/B9), Direct Slave Control (B4), Baseline Response Replay (B2), TCP Pool Exhaustion (T10), Remote Restart (S4), Diagnostic Register Reset (S1) and Broadcast Spoofing (B1);
- capture logs, figures and a console screenshot showing the difference between normal and attack behaviour, and map each result back to the taxonomy cell and control-system objective it affects;
- critically evaluate the paper and explain, honestly, what a software reproduction can and cannot show.

System Setup

The entire testbed is software written by us in Python (standard library plus **NumPy** for arrays and **Matplotlib** for figures; **ReportLab** for this PDF). It needs no PLC, RTU or serial hardware and no external data, so every figure regenerates from a single command. Rather than use an off-the-shelf library such as pymodbus, we implemented the Modbus TCP wire format ourselves — this keeps the project fully self-contained and, crucially, lets the attack toolkit build *malformed and spoofed* frames that a normal client library would refuse to produce.

Background: the Modbus protocol

Modbus uses a simple request/reply model between a *master* (control centre) and one or more *slaves* (field devices). A request carries a one-byte **function code** (read/write coils and registers, diagnostics, etc.) and up to 252 parameter bytes; the slave returns a response with the same function code, or an **exception response** (function code + 128). Unicast slave addresses are 1–247 and address 0 is a **broadcast** to which slaves perform the action but send *no reply*. Modbus TCP (slaves listen on port 502) wraps the same application PDU in a seven-byte **MBAP header** — transaction identifier, protocol identifier (0), length, and unit identifier — giving a maximum 260-byte data unit. The slave performs the passive open and is the “server”; the master is the “client”. None of these messages is authenticated or encrypted, which is the structural weakness the whole taxonomy exploits.

Recreated implementation (custom Modbus TCP stack)

Three components in `src/` make up the testbed:

- `modbus_protocol.py` — encodes/decodes the MBAP header and application PDU for the function codes we use (read/write coils and registers, FC08 diagnostics, FC17 Report Server ID) and builds exception responses; a length-prefixed frame reader implements correct stream framing.
- `modbus_server.py` — a real (localhost) Modbus TCP slave: a multi-threaded TCP listener with a *bounded connection pool*, a datastore (coils, discrete inputs, holding/input registers), a block of FC08 diagnostic counters, and a background thread advancing the physical process. It is deliberately spec-conformant and trusting — no authentication on any function code, broadcast writes accepted silently, FC08 able to restart comms or clear counters — exactly the assumptions the taxonomy relies on.
- `modbus_client.py` — a Modbus TCP master used both as the legitimate SCADA control loop and as the honest baseline.

To give the attacks a *visible physical consequence* (mirroring the paper’s pipeline framing), `process_model.py` simulates a fluid storage tank controlled by the slave: a pump and inlet/outlet valves (coils) drive the tank level, pressure and flow (input registers), with the level held in a safe band by the master’s closed-loop control. The register values are scaled integers (tenths of a percent), exactly how real field devices encode engineering units.

Attack toolkit (testing setup)

`src/attacks.py` implements the representative attacks as functions/classes acting over real TCP sockets against the slave. The man-in-the-middle `MitmProxy` (the paper’s *Rogue Interloper*, B14) sits between master and slave and can passively log traffic (B9), freeze and replay a captured response (B2), or rewrite values in flight. Other functions perform network scanning (B6), Report Server ID reconnaissance (S5), unauthenticated command injection / Direct Slave Control (B4), broadcast spoofing (B1), FC08 Remote Restart (S4) and Diagnostic Register Reset (S1), and TCP connection-pool exhaustion (T10). Because Modbus requires no credentials, none of these attacks needs to defeat any authentication — precisely the point the taxonomy makes.

Experiments

Summary of the original paper.

Huitsing *et al.* group attacks into three categories: those that exploit the protocol specification, those that exploit vendor implementations, and those that target the support infrastructure. The paper (and this project) focuses on the first category — attacks common to *any* spec-conformant Modbus device. Four threats (interception, interruption, modification, fabrication) are analysed against four targets to yield the taxonomies in the paper's Tables 1 (serial) and 2 (TCP); the impact of each attack instance is then summarised against three high-level control objectives — *loss of confidentiality*, *loss of awareness* and *loss of control* (the paper's Table 5). Named attacks include Diagnostic Register Reset (S1), Remote Restart (S4), Slave Reconnaissance (S5), Broadcast Message Spoofing (B1), Baseline Response Replay (B2), Direct Slave Control (B4, whose "fabricate a master" instance B4-3 is called the most serious of all attacks), Network Scanning (B6), Passive Reconnaissance (B9), Rogue Interloper / MITM (B14, fifteen instances) and the TCP-only floods and exhaustion attacks (T9 FIN flood, T10 pool exhaustion, T11 RST flood).

Methodology of the reproduction.

We ran seven scenarios. Each spins up a fresh slave on a local port, then either runs the legitimate master alone (baseline) or launches an attacker thread; every scenario records its numbers into `results/metrics.json` and renders a figure. The scenarios are:

- **E1 — Normal communication.** Master polls and controls the tank, holding the level in a 45–55% band; a passive tap captures and decodes real frames.
- **E2 — Interception (B6, S5, B9).** Unauthenticated network scan, Report Server ID, and passive sniffing of cleartext responses.
- **E3 — Direct Slave Control (B4).** Attacker injects coil writes (close outlet valve, force pump on) to overflow the tank, overriding the master.
- **E4 — Baseline Response Replay (B2).** MITM freezes a "safe" level response and replays it while the true level overflows — loss of awareness.
- **E5 — TCP Pool Exhaustion (T10).** Attacker holds connections until the slave's pool is full and the legitimate master is locked out.
- **E6 — Remote Restart (S4) + Diagnostic Reset (S1).** FC08 abuse to force a comms outage and to wipe the diagnostic counters.
- **E7 — Broadcast Spoofing (B1).** A unit-id-0 write that the slave executes silently with no response.

Code availability.

All code accompanies this report in the `modbus_project/` folder: `src/` (the four modules above), `experiments/run_all.py` (the seven scenarios → all figures + `results/metrics.json`), and `report/build_report.py` (this PDF). The whole suite runs with `python experiments/run_all.py`. The core of the unauthenticated command injection (Direct Slave Control, B4) is just a few lines — no exploit, simply a valid Modbus write:

```
# B4 Direct Slave Control: seize the field device with valid Modbus writes
c = ModbusClient(host, port).connect() # no credential is ever requested
while attacking:
    c.write_coil(OUTLET_VALVE, False) # block the tank's outflow
    c.write_coil(PUMP_RUN, True) # force the inflow pump on
    time.sleep(0.05) # hold the override -> overflow

# B1 Broadcast Message Spoofing: unit id 0 -> slave acts, sends no reply
sock.sendall(build_adu(tid=0x1234, unit_id=0, pdu=pdu_write_single_coil(PUMP_RUN, False)))
```

The suite prints the numeric results captured below; all figures and the metrics file are regenerated on every run.

```
modbus reproduction - python experiments/run_all.py

=====
MODBUS ATTACK TAXONOMY REPRODUCTION - Huitsing, Chandra, Papa & Shenoi (2008)
Self-contained Modbus TCP master/slave + attack toolkit (pure Python)
=====

=== Experiment 1: Normal Modbus communication (baseline) ===

=== Experiment 2: Interception (B6 scan, S5 report-id, B9 passive sniff) ===
B6 scan: device answered on 3 unit-id(s); discovered 4 holding regs + coils with NO authentication
S5 Report Server ID returned: TULSA-RTU-1
B9 passive sniff recovered 3 cleartext response frames (register/coil values readable in the clear)

=== Experiment 3: B4 Direct Slave Control (unauthorized command injection) ===
attacker forced OUTLET_VALVE=closed + PUMP_RUN=on with no authentication
tank overflowed (>=98%) at t=12.1s vs never under normal control

=== Experiment 4: B2 Baseline Response Replay (man-in-the-middle) ===
after replay began, master kept seeing ~49% while true level reached 100%
maximum awareness gap (true - seen) = 50.6 percentage points

=== Experiment 5: T10 TCP Pool Exhaustion (denial of service) ===
attacker connections= 0 active_on_slave= 1 legit master OK (16 ms)
attacker connections= 1 active_on_slave= 2 legit master OK (17 ms)
attacker connections= 2 active_on_slave= 2 legit master OK (3 ms)
attacker connections= 3 active_on_slave= 4 legit master OK (3 ms)
attacker connections= 4 active_on_slave= 5 legit master OK (1 ms)
attacker connections= 5 active_on_slave= 6 legit master OK (16 ms)
attacker connections= 6 active_on_slave= 7 legit master OK (2 ms)
attacker connections= 7 active_on_slave= 8 legit master OK (16 ms)
attacker connections= 8 active_on_slave= 8 legit master REFUSED (2 ms)
attacker connections= 9 active_on_slave= 8 legit master REFUSED (16 ms)
attacker connections= 10 active_on_slave= 8 legit master REFUSED (3 ms)
attacker connections= 11 active_on_slave= 8 legit master REFUSED (15 ms)
pool size=8; first lock-out at 8 attacker connections; slave rejected 7 connection attempts

=== Experiment 6: S4 Remote Restart + S1 Diagnostic Register Reset ===
[attacker] sent FC08/sub01 Remote Restart at t=3.5s
S4 caused ~2.8s of lost communications after the restart
S1 cleared diagnostic counters: bus_messages 85 -> 0

=== Experiment 7: B1 Broadcast Message Spoofing (fabrication) ===
sent broadcast (unit id 0) write PUMP_RUN=OFF: 12 34 00 00 00 06 00 05 00 00 00 00
slave responded to broadcast? False (no response = stealthy, as the paper notes)
PUMP_RUN coil: True -> False (command took effect silently)

=== Building taxonomy coverage + summary figures ===
```

Figure 1. Terminal output from a complete run of the experiment suite, listing the reconnaissance findings, attack effects and DoS sweep.

Results and Discussion

Normal operation baseline (E1)

Figure 2(a) shows the master holding the tank level inside the control band (mean 49.8%, range 44.6–55.1%) by switching the pump — the correct, attack-free behaviour. Figure 2(b) decodes real captured frames: the seven-byte MBAP header followed by the PDU is plainly visible, and every register value and command is in the clear. This cleartext, unauthenticated exchange is the substrate on which all the attacks operate.

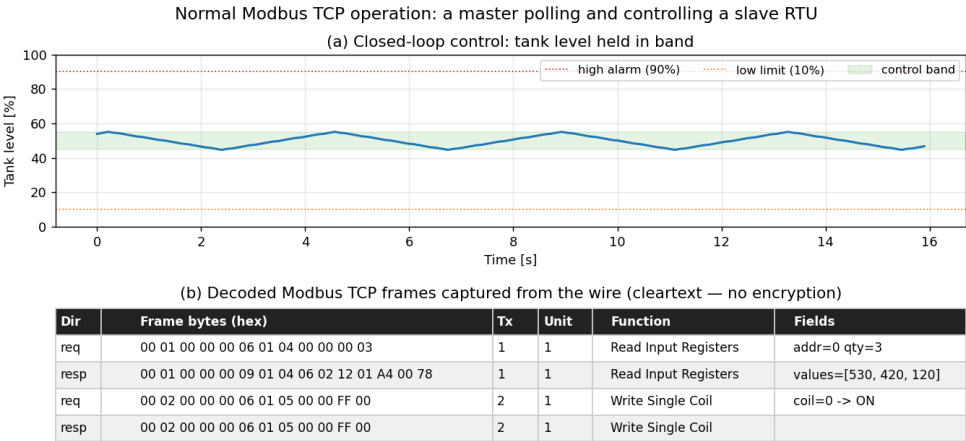


Figure 2. Normal Modbus TCP operation. (a) Closed-loop control keeps the tank level in band. (b) Decoded request/response frames captured from the wire — MBAP header + PDU, entirely in cleartext.

Interception: confidentiality (E2)

With no credentials, an unauthenticated scan recovered the device's full asset map — 4 holding registers and 4 coils with their live values — and Report Server ID (FC17) returned the device identity “TULSA-RTU-1”. The slave answered on every unit identifier probed (3 of 3), confirming there is no unit-level validation. A passive tap recovered 3 cleartext response frames without sending anything at all. This reproduces the paper's Network Scanning (B6), Slave Reconnaissance (S5) and Passive Reconnaissance (B9) attacks, all in the *interception* row of the taxonomy.

Interception attacks: confidentiality lost with no credentials and no cipher

(a) B6 / S5: full asset map recovered by an unauthenticated scan				(b) B9: passive sniff of cleartext responses (values readable on the wire)	
Asset	Address	Value	Meaning	Function	Captured response frame (hex)
Holding reg	0	600	PUMP_SPEED_SP	Read Input Registers	00 01 00 00 00 09 01 04 06 02 15 01 A4 00 78
Holding reg	1	500	LEVEL_SETPOINT	Read Holding Registers	00 02 00 00 00 07 01 03 04 02 58 01 F4
Coil	0	ON	PUMP_RUN	Read Coils	00 03 00 00 00 04 01 01 01 07
Coil	1	ON	INLET_VALVE		
Coil	2	ON	OUTLET_VALVE		
Coil	3	OFF	HIGH_ALARM		
Identity (S5)	FC17	TULSA-RTU-1	device fingerprint		

All payloads are plaintext — an eavesdropper reads every process value and command without breaking any cipher.

Figure 3. Interception attacks. (a) The complete coil/register map and device identity recovered by an unauthenticated scan (B6/S5). (b) Cleartext response frames captured by a passive tap (B9).

Modification & fabrication: Direct Slave Control (E3)

Figure 4 contrasts the two runs. Under normal control the level stays in band (max 55.1%). When the attacker begins issuing unauthenticated coil writes at $t = 7$ s — closing the outlet valve and forcing the pump on — it overrides the master's commands and the tank overflows (reaches 100%) about 5.1 s later. This is the paper's Direct Slave Control (B4): it locks out the master and seizes the field device, the “most serious” attack class (its B4-3 instance fabricates the master) and a direct *loss of control*.

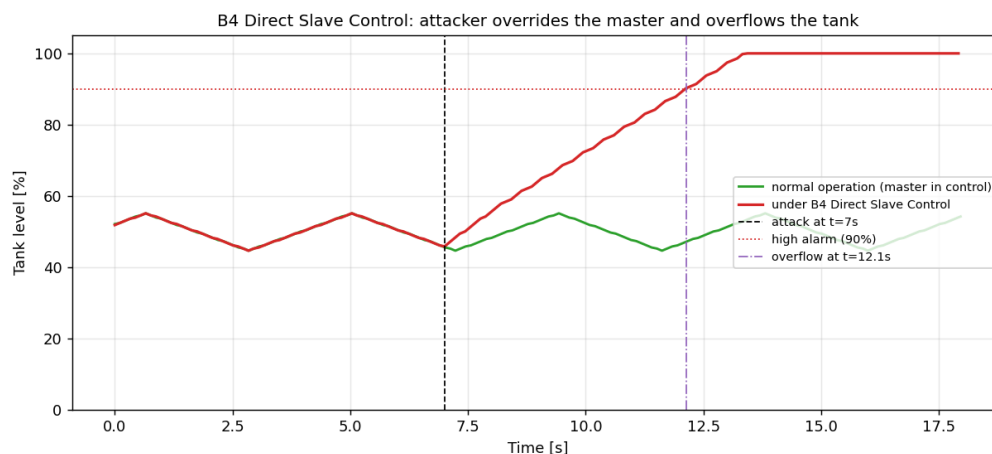


Figure 4. B4 Direct Slave Control. After the attack the operator (green) loses control; the attacker (red) overrides the master and drives the tank past the high-level alarm into overflow.

Modification: Baseline Response Replay (E4)

Figure 5 shows the man-in-the-middle (Rogue Interloper, B14) freezing a genuine “safe” level reading at $t = 6$ s and replaying it to the master while simultaneously seizing the process. The master keeps seeing a steady ~49% and believes everything is normal, while the true level climbs to 100% — an awareness gap of 51 percentage points. This is Baseline Response Replay (B2) and a textbook *loss of awareness*: the operator is blind to a developing incident.

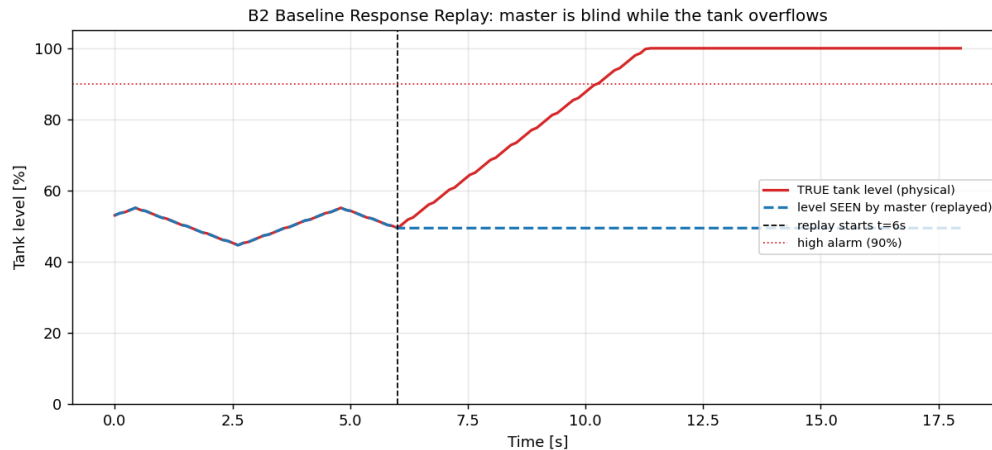


Figure 5. B2 Baseline Response Replay. The level seen by the master (blue, replayed) stays flat and safe while the true physical level (red) overflows — the operator is blinded.

Interruption: TCP Pool Exhaustion (E5)

Figure 6 sweeps the number of attacker-held connections. The slave's connection pool holds 8 clients; as soon as the attacker fills it (at 8 held connections) the legitimate master can no longer connect, and the slave went on to reject 7 further connection attempts. This reproduces TCP Pool Exhaustion (T10), an *interruption* / denial-of-service that needs no malformed traffic at all — merely opening and holding valid TCP connections.

T10 TCP Pool Exhaustion: a bounded connection pool is filled, locking out the master

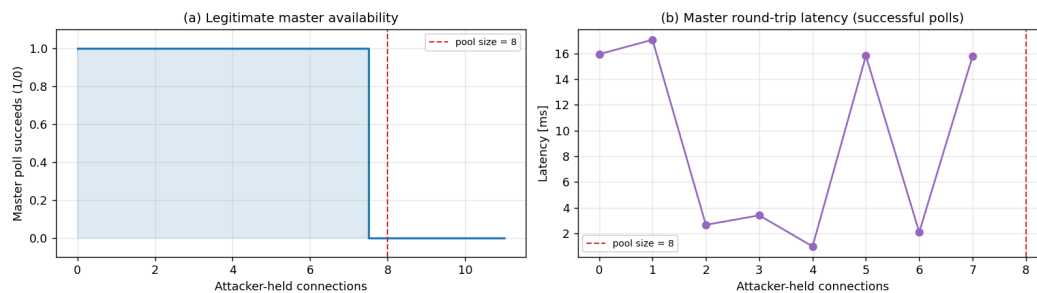


Figure 6. T10 TCP Pool Exhaustion. (a) The master's polls succeed until the connection pool fills, then fail outright. (b) Round-trip latency of the successful polls before lock-out.

Diagnostic abuse: Remote Restart & counter wipe (E6)

Figure 7 shows two FC08 diagnostic attacks. A single Remote Restart (S4, sub-function 0x01) caused a ~2.8 s communications outage during which the slave answered nothing (an *interruption*; repeated, it renders the device permanently inoperable). A Diagnostic Register Reset (S1, sub-function 0x0A) then wiped the slave's counters (bus messages $85 \rightarrow 0$), destroying the very evidence an operator would use to notice the attack — a *modification* with an anti-forensic effect.

FC08 diagnostic abuse: forced restart (availability) and counter wipe (anti-forensics)

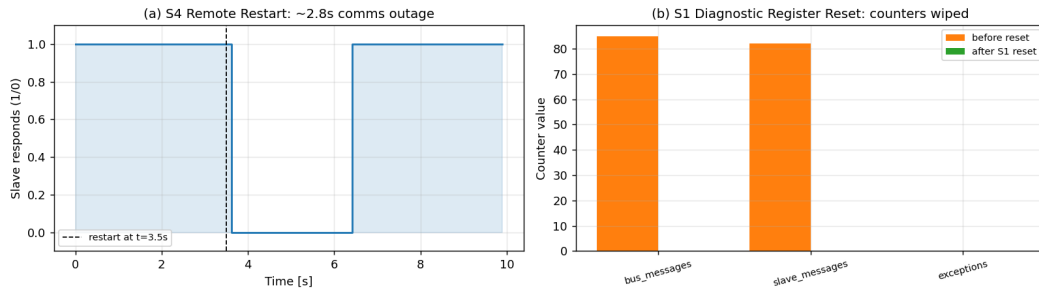


Figure 7. FC08 diagnostic abuse. (a) S4 Remote Restart produces a multi-second comms outage. (b) S1 Diagnostic Register Reset wipes the diagnostic counters to zero.

Fabrication: Broadcast Message Spoofing (E7)

Sending a write to unit identifier 0 (broadcast) changed the PUMP_RUN coil from True to False, and — as the paper notes — the slave returned *no response* (*got_response* = False). The fabricated command therefore leaves no reply trail for a passive monitor to catch, making this *fabrication* attack (B1) particularly stealthy.

Coverage of the taxonomy

Figure 8 places our reproduced attacks back onto the paper's threat×target matrix; Table 1 summarises the measured effect of each. Between them the experiments exercise all four threat categories and three of the four target assets, and all three control objectives from the paper's Table 5 (loss of confidentiality, awareness and control).

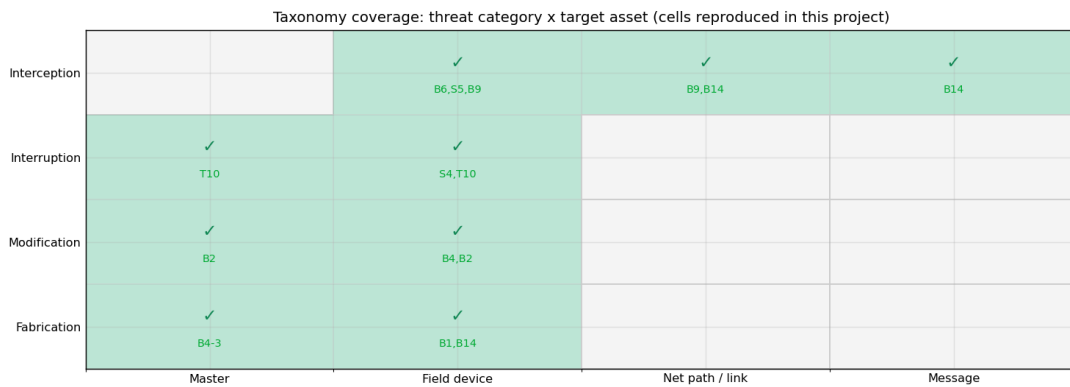


Figure 8. The reproduced attacks mapped back onto the paper's threat-category × target-asset taxonomy (reproduced cells shaded, labelled with the paper's attack designators).

Attack (taxonomy code)	Threat	Target	Measured effect in reproduction
B6 / S5 / B9 Reconnaissance	Interception	Field device	4 regs + 4 coils + identity read; 3 frames sniffed, no auth
B4 Direct Slave Control	Modification	Field device	master overridden; overflow 5.1 s after attack
B2 Baseline Response Replay	Modification	Master	awareness gap 51 pts (true 100% vs seen ~49%)
T10 TCP Pool Exhaustion	Interruption	Master	master locked out at 8 conns (pool 8); 7 rejected
S4 Remote Restart	Interruption	Field device	~2.8 s communications outage
S1 Diagnostic Register Reset	Modification	Field device	counters wiped 85 → 0 (anti-forensics)
B1 Broadcast Spoofing	Fabrication	Field device	coil True → False, no response (stealthy)

Table 1. Summary of the representative attacks reproduced and their measured effect on the testbed.

Comparison with the reference paper

The reference paper reports *no experimental results* — it is a taxonomy — so the comparison is necessarily qualitative: for each attack the paper *predicts* feasibility and an impact class, and the reproduction *confirms* it on a live device and measures the effect. Table 2 lines up the paper's claims with what we observed.

Aspect	Original paper (2008)	This reproduction
Nature of work	Analytical taxonomy, no experiments	Live Modbus TCP testbed + measured attacks
Attacks catalogued / run	20 serial + 28 TCP distinct attacks	8 representative attacks across all 4 threats
Reconnaissance feasible (B6/S5/B9)	Yes (no auth in protocol)	Yes — full asset map + identity recovered
Direct Slave Control (B4) seizes device	Yes; B4-3 'most serious'	Yes — master overridden, tank overflow
Replay blinds the master (B2)	Yes (loss of awareness)	Yes — 51-pt awareness gap
DoS via TCP (T9/T10/T11)	Yes (availability)	Yes — pool exhaustion locks out master
Diagnostic abuse (S1/S4)	Yes (interrupt / reconfigure)	Yes — outage + counter wipe
Broadcast spoofing stealthy (B1)	Yes (no response returned)	Yes — silent write, no reply
Root cause	No auth / integrity / encryption in Modbus	Confirmed: every attack needs zero credentials

Table 2. The paper's predictions next to the behaviour observed in this reproduction.

Critical evaluation

What the reproduction confirms. Every attack we tried worked on the first attempt and required *no* credential, no exploit and, in most cases, no malformed packet — only well-formed Modbus messages used against a protocol that trusts whoever sends them. This empirically validates the taxonomy's central premise and its ranking of severity: interception is trivial, but the modification and fabrication attacks that seize the field device or blind the operator are the ones that cause physical, safety-relevant harm.

Strengths of the paper. The two-axis classification (threat × target) is simple, exhaustive over the protocol surface, and maps cleanly onto control-system consequences (confidentiality / awareness / control), which makes it genuinely useful for risk analysis. Eighteen years on it still describes the attack surface of fielded Modbus accurately, which is itself a striking finding.

Weaknesses and gaps. The paper deliberately covers only specification-level attacks, excluding implementation bugs and infrastructure attacks — the categories that produce many real-world incidents. It enumerates attacks but offers no quantitative risk, no likelihood, and (being from 2008) predates the standard mitigation: *Modbus/TCP Security* (2018) adds TLS and X.509 role-based authorisation, and in its absence the practical defences are network segmentation and deep-packet-inspection IDS such as the DigitalBond/Snort signatures the paper itself references. A reproduction also exposes that some “distinct” attacks are really the same primitive (an unauthenticated write) counted against different assets, which inflates the attack counts.

Limitations of the reproduction

- The testbed is a localhost Modbus **TCP** simulation, not a real PLC/RTU or a serial bus; serial-only electrical/timing attacks are approximated, not physically reproduced.
- We implemented a representative **8 attacks**, not all 48 distinct attacks in the taxonomy. Raw-socket spoofing attacks (T9 FIN flood, T11 RST flood) and Irregular TCP Framing (T5) were not run as live floods — our parser simply rejects malformed frames — with T10 standing in as the DoS representative.
- Absolute numbers are properties of **our** testbed parameters (pool size = 8, a 2 s restart timer, chosen process rates), not measurements of any real device; a real RTU's power-up self-test or pool size could differ by orders of magnitude. Per the course guidance these are testbed limitations, not flaws in the method, and no result is forced to match the paper (which in any case reports no numbers).

- Response Delay (B11) and full MITM modification were only partially exercised; network latency/jitter is not modelled.

Conclusion

We turned an analytical attack taxonomy into a working, self-contained reproduction: a from-scratch Modbus TCP master/slave driving a simulated tank process, plus an attack toolkit. Running one representative attack from every threat category, we empirically confirmed the paper's claims — reconnaissance recovers the whole device with no credentials; Direct Slave Control seizes the field device and overflows the tank; response replay blinds the operator; pool exhaustion denies service; and FC08 diagnostics force outages and erase evidence — and mapped each result back to the taxonomy cell and control objective it harms. Because the reference paper contains no experiments, the reproduction does not chase matching numbers; its contribution is to make the taxonomy *concrete* and to show, hands-on, that Modbus's lack of authentication, integrity and encryption makes every one of these attacks not merely possible but easy. The clear practical lesson — consistent with the paper and with later standards — is that Modbus must be defended by its environment (segmentation, monitoring/IDS, and the modern TLS-based Modbus Security extension), since the protocol cannot defend itself.